

Inventory Strategy

Objective

Produce a machine-readable, LLM-friendly inventory of everything in this directory — clearly separating stock Open WebUI from custom RegOS work — in the fewest tokens possible.

1. What the Inventory Will Contain

The inventory/ folder will hold **four files**:

File	Purpose	Format
MANIFEST.yaml	Master inventory — every meaningful file/folder classified as base line, custom, or modified	YAML
BASELINE.md	What stock Open WebUI 0.8.1 looks like after build/deploy — the reference frame	Markdown
REGOS_MAP.md	What RegOS adds: every custom component, its purpose, dependencies, and which feature it belongs to	Markdown
SOURCE_PATCHES.md	Specifically which stock Open WebUI source files have been modified, what was changed, and how the installer applies those changes	Markdown

Why four files instead of one

- `MANIFEST.yaml` is the programmatic index — an LLM can parse it instantly to answer "what is file X?" or "list all custom scripts."
- `BASELINE.md` gives an LLM the mental model of stock Open WebUI so it never confuses custom work with upstream code.
- `REGOS_MAP.md` is the narrative companion — explains the *why* and *how* that YAML can't capture.
- `SOURCE_PATCHES.md` isolates the highest-risk area (modified stock files) for quick auditing.

2. Classification Rules

Every file/folder in the repo gets exactly one label:

Label	Definition	Examples
baseline	Ships with Open WebUI 0.8.1, unmodified	<code>backend/</code> , <code>src/</code> , <code>Dockerfile</code> , <code>pyproject.toml</code>
custom	Created entirely for RegOS; does not exist in stock Open WebUI	<code>regos_setup/</code> , <code>regos-installer/</code> , <code>RegOS_*.docx</code> , <code>legal-chunking-dashboard/</code>
modified	Stock Open WebUI file that has been changed for RegOS	Svelte components patched by installer, backend routes with guest endpoint
ephemeral	Build artifacts, caches, lockfiles — not inventoried in detail	<code>node_modules/</code> , <code>__pycache__/</code> , <code>.svelte-kit/</code> , <code>uv.lock</code>

3. How Each File Gets Built

3A. `BASELINE.md` — Stock Open WebUI Reference

Method: Do NOT traverse every file. Instead:

1. Read the Open WebUI GitHub README (already have version 0.8.1 from `package.json`)
2. Use knowledge of the repo structure from the git history (upstream commits are visible)

3. Document the canonical directory tree: backend/, src/, static/, docs/, scripts/, cypress/, test/, config files at root
4. Note what gets generated at build time: .svelte-kit/, node_modules/, backend/data/

Token cost: ~500 tokens to write. No file reads needed beyond what we already have.

3B. MANIFEST.yaml — Master Index

Method:

1. Start from the directory listing we already have (3-level tree, ~120 entries excluding node_modules/.git)
2. Classify each top-level entry using the rules above
3. For custom folders (regos_setup/, regos-installer/, gtihub-regos-installer/, legal-chunking-dashboard/, benchmark/), list their contents one level deeper
4. For baseline folders (backend/, src/, static/, etc.), list only the top-level folder — no need to enumerate every stock file
5. For root-level files, classify each individually (we already have the list)

Structure:

```
version: "1.0"
open_webui_version: "0.8.1"
inventory_date: "2026-03-20"

entries:
  - path: "backend/"
    type: baseline
    description: "Open WebUI Python backend (FastAPI)"

  - path: "regos_setup/"
    type: custom
    description: "Core RegOS configuration, functions, data, and scripts"
    children:
      - path: "functions/graphrag_filter.py"
        type: custom
        description: "Core GraphRAG retrieval filter (v0.17.3)"
      # ...

  - path: "RegOS_Architecture.docx"
    type: custom
    description: "RegOS architecture documentation"
```

Token cost: ~2000 tokens. Uses data already collected — no additional file reads.

3C. REGOS_MAP.md — Custom Work Narrative

Method:

1. Source primarily from regos_setup/README.md and regos_setup/REGOS_CHANGELOG.md (both already read)
2. Organize by functional area, not folder structure:
 - **Filter Functions** (graphrag_filter, audit_logger, threshold_eval)
 - **APIs** (breach_api, scada_stream, apas_bridge)
 - **Data Files** (thresholds, mappings, concepts, graph export)

- **Setup & Installer** (regos_setup/setup/, regos-installer/, gtihub-regos-installer/)
 - **Source Patches** (modified Svelte/Python in stock Open WebUI)
 - **Benchmarks** (pdf_benchmark, docling vs GPT-4o results)
 - **Standalone Apps** (legal-chunking-dashboard)
 - **Documentation** (all RegOS_*.docx, .md docs in extra/)
 - **Utility Scripts** (demo_*, diag_*, verify_*, contribution_stats)
 - **Workflow Automation** (n8n escalation workflow)
 - **Test Suites** (adversarial guardrails, MECE test prompts)
3. For each component: one-line purpose, version (if applicable), key files, dependencies

Token cost: ~1500 tokens. No additional file reads needed — everything sourced from already-read docs.

3D. SOURCE_PATCHES.md — Modified Stock Files

Method:

1. Read regos-installer/source-patches/apply-patches.py to understand what files are patched and how
2. Cross-reference with git log (the 4 RegOS commits visible in history)
3. Document each modified stock file: what was changed, why, which installer step applies it

Token cost: ~800 tokens. One additional file read (apply-patches.py).

4. Execution Plan

Step	Action	Reads Required	Output
1	Write BASELINE.md	0 (use existing knowledge)	inventory/BASELINE.md
2	Write MANIFEST.yaml	0 (use existing directory data)	inventory/MANIFEST.yaml
3	Read apply-patches.py	1 file	Understanding of source patches
4	Write SOURCE_PATCHES.md	0	inventory/SOURCE_PATCHES.md
5	Write REGOS_MAP.md	0	inventory/REGOS_MAP.md
6	Write inventory/README.md	0	Index file explaining the inventory

Total estimated reads: 1 additional file (apply-patches.py)

Total estimated writes: 6 files

Estimated token budget: ~6000 tokens of output across all files

5. What We Deliberately Skip

- `node_modules/` — NPM dependencies, thousands of files, zero custom work
- `.git/` — Git internals
- `.svelte-kit/` — Build output, regenerated
- `__pycache__/` — Python bytecode cache
- `.claude/` — Claude Code worktree artifacts
- **Individual stock files** inside `backend/`, `src/`, `static/` — only enumerate the top-level structure
- **File-by-file diffing** of stock Open WebUI — rely on the installer's `source-patches/` as the authoritative record of modifications

6. Design for Future LLMs

Any LLM dropped into this repo should be able to:

1. **Read** `inventory/README.md` → understand what the inventory is and where to look
2. **Parse** `MANIFEST.yaml` → instantly classify any file as baseline/custom/modified
3. **Read** `BASELINE.md` → understand what stock Open WebUI provides
4. **Read** `REGOS_MAP.md` → understand everything built on top
5. **Read** `SOURCE_PATCHES.md` → understand exactly which stock files were touched and why

This is the "5-minute onboarding" for any future agent or developer.